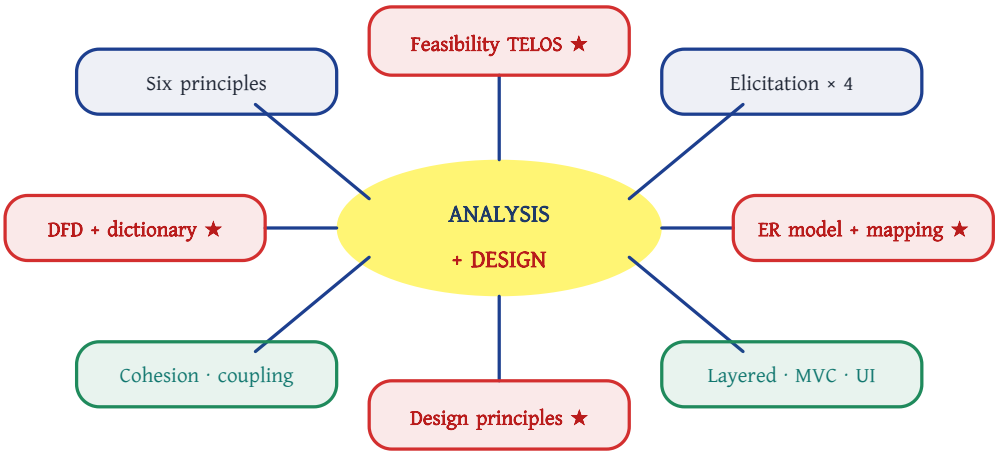
Analysis, Modeling, Design & Architecture

Requirements · DFD · ER modelling · design principles · architecture — in 12 sheets



↑ the whole topic on one page: red = high-yield, green = applied

Syllabus coverage

"Analysis Concepts and principles: requirement analysis, **Analysis modeling**, **data modeling**. **Design concepts and principles**, **Architectural design**, User Interface design". Use cases, the Unified Process and the full UML treatment are in the Use Cases / OO Design notes.

High-yield list

① analysis = what, design = how
② context diagram = one process
③ DFD balancing rule
④ functional cohesion is best
⑤ data coupling best, content worst
⑥ M:N → a new relation, 2 FKs

Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

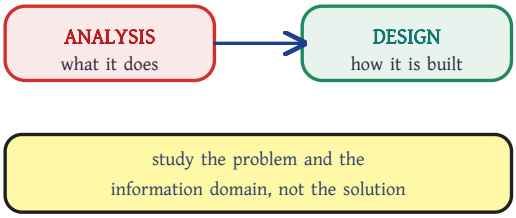
Contents

✓ Analysis & six principles … p. 2
✓ Functional vs non-functional; SRS … p. 2
✓ Elicitation: four techniques … p. 3
✓ Feasibility TELOS & validation … p. 3
✓ DFD: elements, context, balancing … p. 4
✓ Data dictionary + library DFD … p. 5
✓ ER: entities, attrs, relations … p. 6
✓ Library ERD → relational schema … p. 7
✓ Cohesion, coupling, principles … p. 8
✓ Design types, architecture, UI … p. 9–10

1 Analysis: the "What" Before the "How"

Definition: **Software analysis** = **understanding and specifying what the software must do**, before design and implementation. It studies the problem and the **information domain**, never the solution. Analysis answers **WHAT**, design answers **HOW**, implementation is the doing.

Principle	What it demands
Correctness	every real need captured
Consistency	no two requirements clash
Completeness	functional + non-functional
Feasibility	achievable within limits
Traceability	traced to source and test
Understandability	clear to every reader



the sentence that separates the two phases

2 Functional vs Non-Functional Requirements

	Functional	Non-functional
Answers	which services and behaviours	how **well** and how **much**
Examples	log in, search, issue a book	reply in 2 s; 99.9 % uptime; AES-256
Interested party	the end user, the analyst	the user **and** the organisation

Memory trick: Three levels: **business** "cut overdue returns" → **user** "staff see a due-date list" → **system** "the query returns in 2 s".

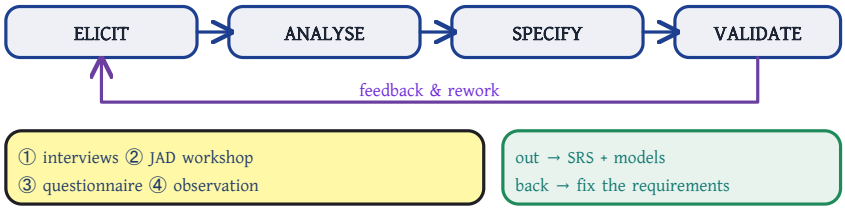
Exam trap: "The system shall be **fast**" is not a requirement — it is unverifiable. Every non-functional need needs a measurable criterion.

3 Specification: What the SRS Must Contain

SRS (IEEE 830 / ISO-IEC 29148): **1 Introduction** — purpose, scope, definitions, overview, references · **2 Overall description** — perspective, functions, user classes, environment, constraints, assumptions · **3 Specific requirements** — external interface, functional, performance, design constraints, system attributes · **4 Appendices** — glossary, index.

4 Requirements Elicitation: the Four Techniques

Technique	Strength	Weakness
Interview	deep detail; instant follow-up	slow; costly; interviewer bias
Workshop / **JAD**	fast; builds consensus; ownership	needs a skilled facilitator
Questionnaire	cheap; repeatable; wide coverage	low response; no clarification
Observation	captures the workflow **used**	slow; users behave differently



requirements engineering is a loop, not a line — validation sends work back

5 Feasibility Study: TELOS

- **T**echical — can it be built at all with the available hardware, software and skills?
- **E**conomic — cost–benefit: do the benefits exceed the costs? (payback period)
- **L**egal — law, privacy, licensing, contracts, standards
- **O**perational — will it fit the way the organisation works?
- **S**chedule — can it be delivered in the time available?

Memory trick: Say it as a word: **T-E-L-O-S** — Greek *telos* = the goal. The report ends in a **go / no-go** call.

Exam trap: Feasibility is judged **before** the SRS and again when the model is built — it is not the last step.

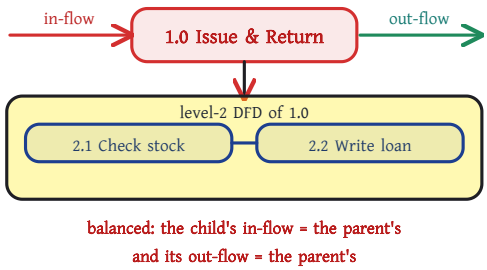
Validation: Proving the model and spec are right: reviews and inspections, prototypes, acceptance test cases, model walkthrough, decision tables. Check correctness, completeness, consistency, realism, verifiability.

Worked judgement ✍️: A hospital wants online appointments. **T**: a server and a biometric device must be bought. **E**: six staff-years saved, licences cost three, payback five years? **L**: patient data must stay in the country. **O**: counter staff will not re-key everything. **S**: go-live before the quarter closes? Only T, L and O clear — the project waits.

6 The Analysis Model and the DFD

Analysis model: an **abstract representation of the system** built from the requirements. **DFD = what happens** (functions and flows) · **ERD = what exists** (data structure) — complements, never rivals.

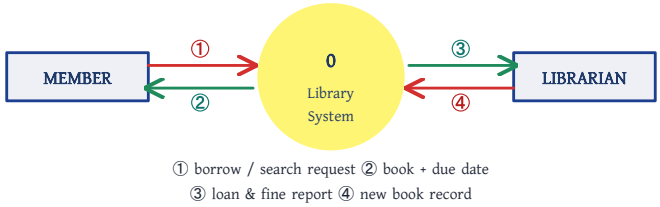
Element	Symbol	Meaning
External entity	square	a source or sink outside the system
Process	circle	turns input data into output data; numbered 1.0, 2.0 …
Data store	two lines	related data at rest, inside the system
Data flow	arrow	a **named data element** moving between elements



balancing: level n+1 replaces one parent process, flows unchanged

7 Level 0: the Context Diagram

Level 0 = context diagram ★: The **whole system as one process**, joined to its external entities by the flows crossing the system boundary. **No store, no internal process** — level 1 adds those.



frequently tested: one bubble = the entire system

Exam trap: It shows neither the database tables (that is the ERD), nor the internal processes (level 1), nor timed messages.

Levels & decomposition: Context (0) → level 1 = one bubble per major function → level 2, 3 … Numbering: 1.0, 2.0, then 2.1, 2.2. Complexity is handled by **divide and conquer**.

Rules (DFD): No store → store, no store → entity, no entity → store. Every process needs ≥ 1 input and ≥ 1 output (a grey or black hole is an error). Flows are **logical** and **named**.

8 The Data Dictionary

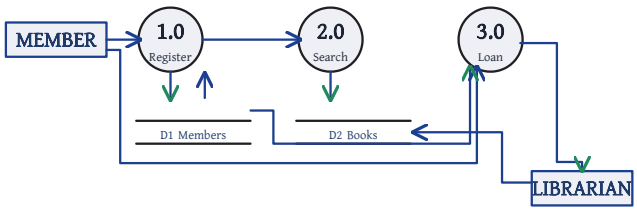
Definition: The **specification of every data flow, data store and data element** in the DFD. Each entry has four parts: **name = composition / definition / where used / comment**. An entry that cannot be written down is a requirement that is still fuzzy.

Symbol	Means
x = a + b	x is **composed of** a and b
x = a, b	alternatives — a **or** b
x = [a]	repeated **one or more** times
x = {a}	optional — **zero or one**
qualifier/x	x is the key that identifies one y
* text *	comment / alias

Entries for case A ✍️:

Loan = loan_id + member_id +
accession_no + issue_date +
due_date + {return_date}
Member = member_id + name +
phone + membership_type
Book = accession_no + isbn + title + author

Case A — the same library system, drawn as a DFD



level 1: three processes, two stores, two external entities

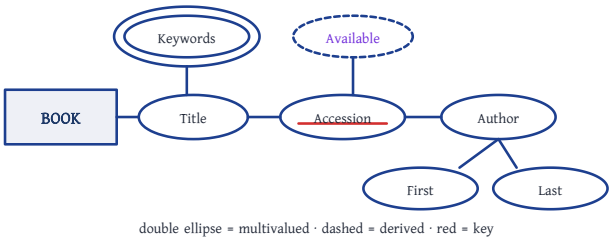
- **1.0 Register**: member data → D1 Members.
- **2.0 Search**: catalogue query → book list; reads D2.
- **3.0 Loan**: issue / return → checks D2, writes D1, returns book + due date.
- **Librarian**: maintains D2, receives the overdue report.

Balanced against the context diagram of sheet 4: every flow entering the system there is consumed by a process here, and every flow leaving there is produced here.

Exam trap: A DFD shows **logical** flow, so a "book file" is not "C:\library\books.txt". If the question asks for the database design, draw the ERD.

9 Entity–Relationship Modelling

Definition: The **ER model** (Chen, **1976**) is a **conceptual data model**: it pictures the real world as **entities**, the **attributes** they carry and the **relationships** between them, with no reference to any DBMS, table or file.

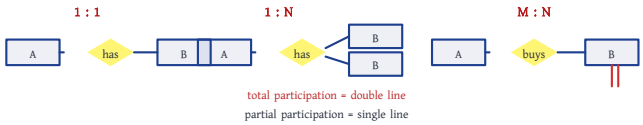


attribute notation: simple, key, composite, multivalued, derived

Entity: a real-world object or event that can be distinctly identified. A **strong** entity owns its key; a **weak** entity (double rectangle, partial key) has none of its own and is identified only through its **owner**.

Attributes: **simple / composite** (name = first + last) · **single / multivalued** (several phone numbers) · **derived** (age from date of birth) · **key** (underlined, minimal).

Relationships: an association among entities. **Degree** = how many entity types take part: unary (1), binary (2), ternary (3). Unary is also **recursive** — Employee *manages* Employee.

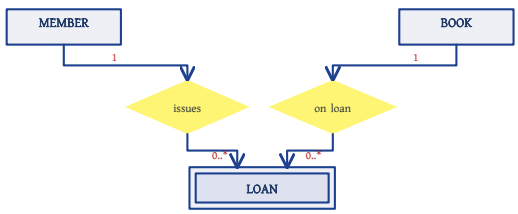


cardinality (how many at most) and participation (must it join?)

Cardinality	Library example
1 : 1	one member ↔ one card
1 : N	one member → many loans
M : N	members ↔ books, over time
Total part.	every instance **must** join
Partial part.	an entity **may** stay out

Exam trap: Cardinality and participation are different questions: "how many at most?" vs "must it take part?". 1 : N does **not** make the N side mandatory.

10 Case B — the ER diagram, then the relational schema



Chen notation — LOAN is weak: key = Member_id + Accession_no

A **MEMBER** issues 0..* **LOAN**s and a **BOOK** appears in 0..* **LOAN**s; LOAN joins both totally, so it cannot exist without both. It carries Issue_date, Due_date and {Return_date}. When the same two entities meet again later in time — loans, orders, enrolments — you need a **third relation holding the event.**

> **Exam trap:** MEMBER–LOAN is **not** a plain 1:N on one line: the pair (member, book) repeats over time, so it is **M:N**, resolved by making LOAN the **associative entity**.

11 ER → Relational Mapping: the Seven Rules

#	Source construct	Relation produced
1	Strong entity	same attributes; its key = **primary key**
2	Weak entity	owner key as FK; PK = owner key + partial key
3	1 : 1 relationship	one side's key as FK in the **other** relation
4	1 : N relationship	key of the **one** side as FK in the **many** side
5	M : N relationship	**new relation**: the two keys as FKs; the pair is the PK
6	Multivalued attribute	**new relation**: entity key + attribute = the PK
7	N-ary relationship	**new relation**: all participants' keys = the PK

Case B applied ✍️: *MEMBER(member_id, name, phone)*
BOOK(accession_no, isbn, title)
LOAN(member_id, accession_no, issue_date, due_date, {return_date})
AUTHOR_BOOK(isbn, author_name)
rule 2 gives the LOAN key; rule 6 splits a multi-author title.

Data model	Answers
Conceptual	which entities and relationships exist — the ERD
Logical	attributes, domains, keys; mapped to the relational model
Physical	tables, columns, data types, indexes — DBMS-specific

> **Exam trap:** Conceptual → logical → **physical**. The conceptual model has **no attributes and no keys.**

12 Design: Turning Requirements into a Blueprint

Principle	In practice
Abstraction	describe **what** a unit does, hide **how**
Modularity	split into separately testable **modules**
Information hiding ★	(**Parnas, 1972**) hide what is **likely to change**
Separation of concerns	one module, one concern
Refinement	top-down, step by step to code
Reuse	reuse an existing design or component

Why it pays: Modules are understood, unit-tested, reused and changed independently; a fix stays inside one module.

Exam trap: **Abstraction ≠ encapsulation.** Abstraction decides what to expose; encapsulation bundles data with its operations. **Modularity is the goal, information hiding the technique.**

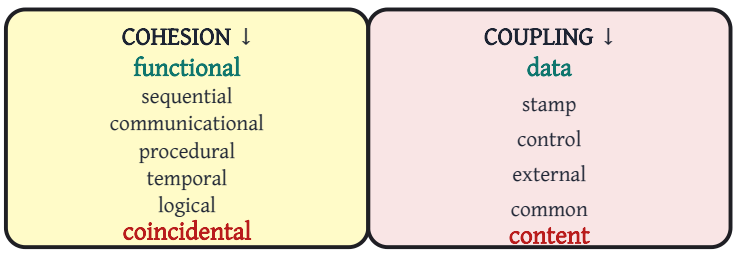
13 Functional Independence: Cohesion & Coupling

Cohesion: how strongly the **elements inside one module** belong together for one purpose. **Higher is better.**

#	Type	Ask
1	**Functional ★**	one clear job?
2	Sequential	output → input?
3	Communicational	same data in-out
4	Procedural	fixed order?
5	Temporal	run together only
6	Logical	any one chosen?
7	**Coincidental**	same file only

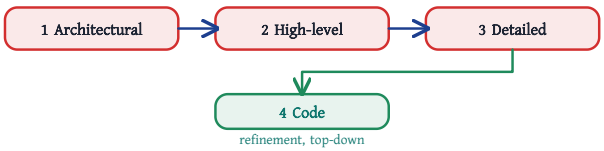
Coupling: how strongly one module **depends on** another. **Lower is better.**

#	Type	Shared
1	**Data ★**	simple parameters
2	Stamp	a whole structure
3	Control	a control flag
4	External	an interface
5	Common	a global area
6	**Content**	internal data



both lists are examined directly — learn them in this order

14 Three Levels of Design

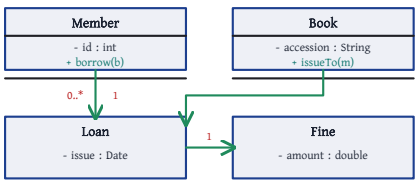


each level refines the one above it

Level	Decides
1 Architectural	the system structure: components, their relationships and constraints; the data architecture and interfaces
2 High-level (modular)	modules → submodules; which data structures and abstract data types each module owns
3 Detailed	the exact algorithm, nesting of records, language-level data structures

15 The Architectural Design Process

1. Address the design: **which functions** must the design model perform?
2. Design the data — the domain model becomes the **design class diagram** (DCD)
3. Assign functions to components: map each class onto a component or subsystem
4. Refine the **design classes**: attributes, operations, associations and **navigability**
5. Define the **design sequence**: the **order in which classes are implemented and unit-tested**



case C: domain classes → design classes

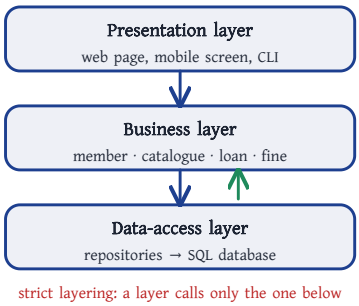
- The **domain model** (data only) gains **operations**, **navigability arrows** and packages.
- **Case C**: Member 1 → 0..* Loan, Book 1 → 0..* Loan, Loan 1 → 0..1 Fine.
- Design sequence: **Book** and **Member** first (they carry the data), then **Loan**, then **Fine**; unit-test each with its collaborators.
- The named GRASP patterns do exactly this job.

Information Expert, Creator, Low Coupling–High Cohesion

> **Exam trap:** The DCD is **not** the domain model: it is implementation-aware, while the domain model holds only classes, attributes and associations.

16 Architectural Styles

Style	How the components talk	Example
Layered	serves the layer above, uses the one below	UI → business → data
Pipe-and-filter	a stream through a chain of filters	Unix shell: cat f | grep x
Client–server	a client requests services	browser and web server
Repository	components read/write one store	shared database
Event-driven	components announce events	GUI listeners, queues
Broker	a broker matches request and service	service lookup, CORBA bus
MVC	Model data, View render, Control events	web frameworks, desktop apps
Microservices	one service per business capability	separate payment service



the library system as a three-layer architecture

Why layered wins ✍️: Move the database from MySQL to PostgreSQL and only the data-access layer changes; restyle the UI and only the top layer changes. Each layer is unit-tested alone; the price is a copy of the data object at every boundary.

Memory trick: MVC: **M**odel = what is true · **V**iew = what it looks like · **C**ontroller = what the user just did.

17 UI Design: Principles for a Usable Interface

- **Visibility of system status** — show what is happening.
- **Match the real world** — the user's language, their units.
- **User control and freedom** — undo, cancel, a clear exit.
- **Consistency and standards** — one look, one meaning.
- **Error prevention** — constrain, confirm, offer a default.
- **Recognition over recall** — show the options.

Goal: intuitive, efficient, pleasant use. Three **golden rules: place the user in control, reduce the user's memory load, make the system consistent.**

Exam trap: **Usability ≠ learnability.** A system can be learnable and still not usable; a novice-friendly system can be unusable for an expert doing the same job all day.

- **Accessibility** — usable by elderly and disabled users.

Night-Before-the-Exam Sheet

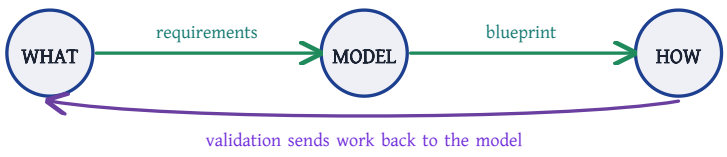
Fact sheet:

Analysis = **what** · design = **how**	6 principles: correct, consistent, complete
...feasible, traceable, understandable	Elicitation: interview, JAD, survey, observe
TELOS = T, E, L, O, S feasibility	DFD: entity, process, store, flow
Level 0 = context = **1 process**, no store	Balancing: child's flows = parent's flows
Levels: 1.0, 2.0 → 2.1, 2.2 ...	Cohesion: functional → coincidental
Coupling: data → content	ER: entity, attribute, relationship
Degree = number of participants	Double line = total participation
M:N, multivalued, n-ary → new relation	Conceptual → logical → physical
Design: architectural → high → detailed	Layered, MVC, client–server, events

1. Context diagram = one process
2. DFD never links store → store
3. Every process needs in and out
4. Functional cohesion is best
5. Data coupling is the loosest
6. Content coupling is the worst
7. M:N becomes a junction relation
8. Weak key = owner key + partial
9. Double line = total participation
10. Conceptual model has no attributes
11. Information hiding: Parnas, 1972
12. Microservices deploy independently

Top traps: "what" and "how" swapped · a context diagram drawn with data stores · a flow read as physical storage · cohesion and coupling ranks reversed · an M:N left as two columns · total participation read as cardinality · attributes given to the conceptual model

All tricks together: context = one bubble · balance in against out · number every process · the flow must carry a name · weak = double box + dashed diamond · underlining = key, double ellipse = many, dashed = derived · strict layering calls downward only · control, memory load, consistency



requirements → model → design, and back again

Also practise (written): ① Context + level-1 DFD for a hospital, with balancing · ② A data dictionary for six flows · ③ ER → relations for a bank with M:N accounts

Where the neighbours live: Use cases, the Unified Process and the UML by type sit with the Use Cases / OO Design notes; GRASP patterns sit with the GRASP notes.

✓ Practice MCQs (exam pattern, 10 marks)

1. Loosest coupling is:
(a) Data coupling (b) Content
(c) Common (d) Control

2. Strongest cohesion is:
(a) Coincidental (b) Functional
(c) Temporal (d) Logical

3. Level 0 DFD shows:
(a) Processes (b) The tables
(c) Messages (d) One process

4. Not allowed in a DFD:
(a) Proc → store (b) Proc → entity
(c) Store → store (d) Entity → proc

5. Weak entity key is:
(a) Partial key (b) Owner + partial
(c) Surrogate key (d) Owner key only

6. M : N maps to:
(a) A new relation (b) Nothing
(c) A new column (d) A procedure

7. Double line means:
(a) Partial part. (b) Ident. rel.
(c) Total part. (d) Multivalued

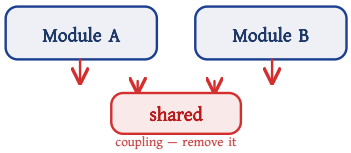
8. Multivalued maps to:
(a) An array (b) Two columns
(c) A derived (d) A relation

9. UI / business / data:
(a) Layered (b) Repository
(c) Broker (d) Pipe-filter

10. "Can it be built?":
(a) Legal (b) Economic
(c) Schedule (d) Technical

Answer key → 1-a · 2-b · 3-d · 4-c · 5-b · 6-a · 7-c · 8-d · 9-a · 10-d

Written question: What are cohesion and coupling? Why should a good design have high cohesion and low coupling? List the types of each from best to worst. 10 marks



high cohesion inside, low coupling between

1. **Cohesion** = how much one module's elements belong together; **coupling** = how much modules depend on each other. 3
2. A good design keeps modules **easy to understand, test, reuse and change**: a change in one hardly affects the others. 2
3. Cohesion, best → worst: **functional, sequential, communicational, procedural, temporal, logical, coincidental.** 2
4. Coupling: **data, stamp, control, external, common, content.** 3

Before the exam ✓

- ✓ Can state the six principles of analysis
- ✓ Can name the four elicitation techniques
- ✓ Can draw an ER diagram and map it to relations
- ✓ Can recite both cohesion and coupling ranks
- ✓ Can name the three levels of design

✓ Can draw a context diagram and a balanced DFD